📊 Output Analysis

The Hospital Patient Management System is a menu-driven C++ application that uses a **Doubly Linked List** to manage patient records dynamically. It supports patient registration, record display, searching by Patient ID, and patient discharge by deleting the corresponding record.

1. Main Menu

When the program starts, it displays the following menu:

```text
====================================
 HOSPITAL PATIENT MANAGEMENT SYSTEM
====================================
1. Add Patient
2. Display Patients
3. Search Patient
4. Discharge Patient
5. Exit
====================================
Enter your choice:
```

2. Add Patient

When the user selects option `1`, the program accepts the patient's ID, name, age, and disease.

Sample Output:

```text
Enter your choice: 1

Enter Patient ID: 101
Enter Patient Name: Rahul Patil
Enter Age: 45
Enter Disease: Fever

Patient added successfully!
```

The new patient record is inserted at the end of the doubly linked list. The `prev` pointer connects the new node to the previous patient, while the `next` pointer maintains forward traversal.

3. Display Patients

When the user selects option `2`, the program traverses the linked list and displays all registered patients.

Sample Output:

```text
========== PATIENT RECORDS ==========

Patient ID : 101
Name       : Rahul Patil
Age        : 45
Disease    : Fever
-------------------------------------
Patient ID : 102
Name       : Priya Sharma
Age        : 32
Disease    : Migraine
-------------------------------------
```

Patient records appear in the order in which they were registered.

4. Search Patient

When the user selects option `3`, the program searches for a patient using their Patient ID.

Sample Output — Patient Found:

```text
Enter Patient ID to search: 101

Patient Found!
Patient ID : 101
Name       : Rahul Patil
Age        : 45
Disease    : Fever
```

If the entered Patient ID does not exist, the program displays:

```text
```

Enter Patient ID to search: 999

Patient not found!
```

The search operation uses linear traversal to find the matching patient record.

### 5. Discharge Patient

When the user selects option `4`, the program searches for the specified Patient ID and deletes the corresponding node from the doubly linked list.

**Sample Output:**

```text
Enter Patient ID to discharge: 101

Patient discharged successfully!
```

If the Patient ID is not found, the program displays:

```text
Enter Patient ID to discharge: 999

Patient not found!
```

After deletion, the program updates the neighbouring nodes' `prev` and `next` pointers to maintain the linked list.

Note: In the current implementation, discharging a patient removes their record from the list rather than retaining a discharge status or discharge history.

6. Exit Program

When the user selects option `5`, the program terminates.

```text
Enter your choice: 5

Program ended.
```
```

🔍 Complexity Analysis

Operation	Technique	Time Complexity
Add Patient	Traverse to the end and insert	O(n)
Display Patients	Linked list traversal	O(n)
Search Patient	Linear search	O(n)
Discharge Patient	Search and update pointers	O(n)

Here, \(n\) represents the number of patient records currently stored in the list. Pointer updates during deletion take \(O(1)\) time after the target node has been found.

✅ Conclusion

The program demonstrates how a **Doubly Linked List** can be used to manage hospital patient records. It supports dynamic insertion, traversal, searching, and deletion while maintaining links between adjacent nodes.

The project provides a simple demonstration of pointers, structures, dynamic memory allocation, and linked-list operations in C++.

Limitation: Patient records are stored only in memory and are not saved permanently. The current discharge operation deletes patient information instead of preserving a discharge history.